

- 01 这套系统是什么
- 02 完整链路图
- 03 七个阶段速览
- 04 分镜脚本是怎么做出来的
- 05 归档目录规范
- 06 术语表

创作者使用说明 · SHORT-DRAMA PIPELINE

从一句选题，到一集能剪的成片

这套系统不是一个"一键出剧"的黑箱，而是七个各司其职的 Claude Code 技能 (skill) 按固定顺序接力：上一步产出的 Markdown/JSON/图片，就是下一步的输入。所有产出统一落在 `output/<剧名>/` 目录下，既是工作记录，也是可以随时回头查阅的资产库。

这份文档重点拆解你问到的那部分——**分镜脚本 是怎么一步步做出来的、它什么时候真正去"资产库"里取东西、做完之后 归档在哪里**——同时给出整条链路的全貌，方便你知道自己现在在 哪一步、下一步该找谁。

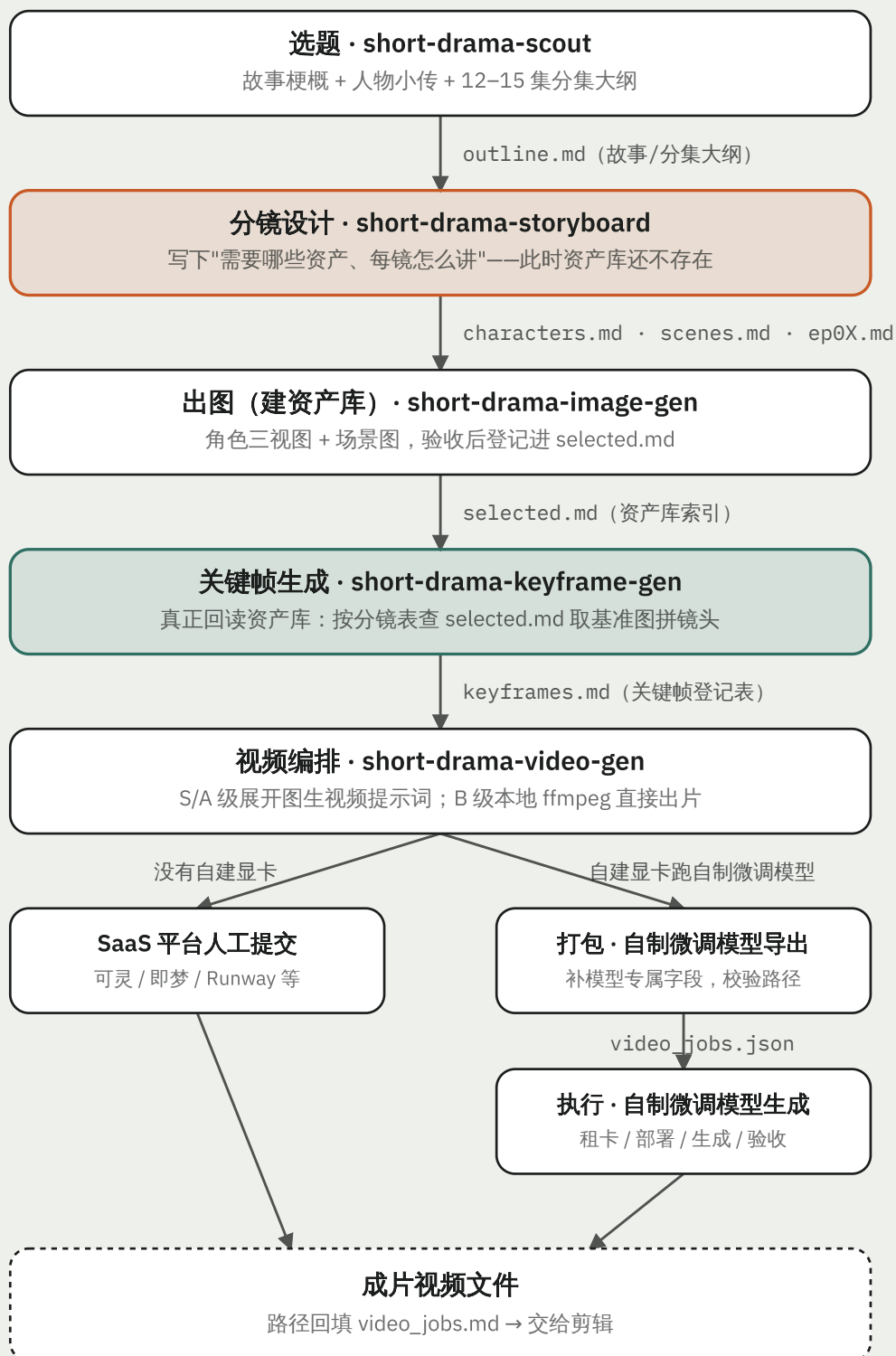
01 这套系统是什么

先建立一个总印象：整部剧的制作过程被拆成 "选题 → 分镜 → 出图 → 关键帧 → 视频" 五个主环节，外加自建显卡时才会用到的两个打包/执行环节。每个环节对应一个技能，技能之间不共享"记忆"，全靠约定好的文件互相交接——这也是为什么下面每一步都会强调"归档到哪里"：文件路径就是环节之间唯一的接口。

一条经验规则：任何一步如果发现"上游该给的文件没给全"，正确做法是停下来回去补齐，而不是凭空猜一个设定往下做——因为下游会按文件里写的路径、措辞原样引用，猜错的东西会一路传递下去，返工成本比多问一句话高得多。

02 完整链路图

竖直方向是制作顺序；每条箭头旁标的是"上一步交给 下一步的具体文件"。第五步之后会分岔——如果你在某个 SaaS 平台（可灵/ 即梦/Runway 等）手动生成视频，走左边；如果你自己租显卡部署了自制 微调模型，走右边，多两个自动化的打包/执行环节。



橙色是「分镜设计」——决定这部剧需要哪些资产、每镜怎么讲；青色是「关键帧生成」——这才是真正回头翻资产库索引、按分镜表取图拼镜头的一步。两者常被混为一谈，但发生在不同阶段。

■ 定义资产需求 / 分镜表 ■ 回读资产库 / 拼具体镜头

03 七个阶段速览

按顺序查表就知道"这一步该找哪个技能、要准备什么、做完东西存在哪"。

阶段	负责技能	需要的输入	产出	归档
① 选题	scout	方向/关键词（可不给，自主选题）	故事梗概、人物小传、分集大纲	对话需求才
② 分镜设计	storyboard	①的故事/分集大纲	style_bible.md characters.md scenes.md ep0X.md	out >/s
③ 出图建库	image-gen	②的 characters.md scenes.md	角色三视图/场景图 + selected.md + manifest.append.jsonl	out >/a
④ 关键帧生成	keyframe-gen	②的 ep0X.md + ③的 selected.md	每集关键镜合成图 + keyframes.md	out >/k
⑤ 视频编排	video-gen	④的 keyframes.md + ②的运镜/分级列	video_jobs.md（S/A级提交清单）+ B级本地视频	out >/v
⑥ 模型打包 仅自建显卡分支	model-export	⑤的 video_jobs.md	video_jobs.json + model_remote_config.json	out >/v
⑦ 模型执行 仅自建显卡分支	model-run	⑥的 video_jobs.json	真正生成的视频文件	下载 vic 填

04 分镜脚本是怎么做出来的

这里拆解你问的核心问题。答案分两半：**分镜表制作的时候，资产库其实还不存在**——它在"定义"资产库该长什么样；真正"回头去资产库里取东西拼镜头"的，是再往后一步的关键帧生成。下面按三个小节讲清楚这条链。

A. 分镜表制作 —— 这时还没有资产库

负责技能：`short-drama-storyboard`。输入只是故事/分集大纲的纯文字，不依赖任何图片。它做的事情是把文字故事拆成一份"下游可以直接拿去出图/出视频，不用回来再问"的详细分镜表，内部按顺序走这几步：

- 1 **定画风锚点与叙事基调** —— 从题材/情绪基调选一个视觉风格（写实/动漫/半写实），写成一句"画风锚点"，后面所有角色、场景、镜头的提示词开头都必须原样复用这句话，避免中途风格漂移。落在 `style_bible.md`。
- 2 **定分级方法论 (S/A/B/C)** —— 不是每个镜头都值得花钱精细生成：S 级（高光动作/反转瞬间，一集 2-4 个）、A 级（对话/反应镜）、B 级（过渡/空镜，用运镜特效代替）、C 级（直接复用素材）。这个分级会一路带到出图、关键帧、视频编排每一步，决定花多少预算。
- 3 **建资产清单** —— 通读故事列出"人物清单"和"场景清单"，标注首次登场集数、造型/日夜版本。这一步是资产库的"需求清单"，不是资产库本身——此时一张图都还没生成。
- 4 **产出角色/场景提示词** —— 对清单里每个角色/场景写一份可直接喂给图像模型的提示词（含画风锚点、外观/服装、常用表情差分），存进 `characters.md / scenes.md`。这两份文件是下一阶段"出图"的原材料。
- 5 **逐集逐镜设计分镜表** —— 核心产出。每一镜写满 14 个字段：镜号、场景编号、出场人物、画面描述、景别、运镜、时长、台词/旁白、音效/BGM、分级(S/A/B/C)、图像生成提示词、运动描述、转场、备注（是否复用素材）。存进 `ep01.md ep02.md ...`。

归档位置：`output/<剧名>/storyboard/style_bible.md、characters.md、scenes.md、ep01.md...ep0N.md`。这一批文件是给人看、也是给下一阶段脚本直接引用的"设计蓝本"，可以正常提交进版本库。

B. 资产库怎么生成出来

负责技能：`short-drama-image-gen`。它读取上一步的 `characters.md / scenes.md`，把里面的提示词组装成 `jobs.json`（角色固定生成"正面+侧面+背面"三视图合一的图，场景

按时段生成 3-4 张变体)，调用出图脚本批量生成，Claude 逐张实际看图验收，选出可用的之后登记进一张表：

output/<剧名>/assets/selected.md 节选（真实项目产出）：

角色 差分/版本	选中文件路径
--- --- --- ---	
江砚 出狱期·正面	assets/江砚_出狱期_正面/江砚_出狱期_正面_00.png
江砚 出狱期·侧面转身	assets/江砚_出狱期_侧面转身/江砚_出狱期_侧面转身_00.png

这张 selected.md 就是**资产库真正的索引**——它把 A 阶段里"文字设定"变成了"可以被程序按名字查到的实际文件路径"。同时每张图的生成记录（用的哪条 prompt、参考图、时间戳）追加进 manifest.append.jsonl，方便回溯。

归档位置：output/<剧名>/assets/<job_id>/<job_id>_00.png ... + selected.md + manifest.append.jsonl。

C. 真正"从资产库调取信息"的一步 —— 关键帧生成

负责技能：`short-drama-keyframe-gen`。这一步才是你问题里"分镜脚本制作如何从资产库调取信息"最贴切对应的环节——它不再凭空写提示词，而是同时打开两份文件对照着做：

输入必须齐全：A 阶段的 ep0X.md（分镜表）+ B 阶段的 selected.md（资产库索引）+ characters.md/scenes.md（用于核对造型/时段版本）。缺哪样就先回去补，不能凭空猜一张图当基准。

- 1 **挑关键镜头** —— 从这一集分镜表里挑约 8-12 个真正关键的镜：开场钩子、结尾钩子、所有 S 级镜、核心冲突相关的 A 级反应镜；标了"复用素材"或纯空镜的直接跳过，不重复出图。
- 2 **展开提示词 + 查资产库取路径** —— 把分镜表里的简写提示词 展开成完整 prompt，再按这一镜的人物/场景/造型/时段，去 selected.md 里逐行核对着**查出对应的基准图文件路径**——这一步"查表取路径"就是"调取资产库信息"的具体动作。查不到对应素材就停下来告诉用户缺哪张，不用文字硬猜替代。
- 3 **组装任务并生成** —— 查到的路径填进 jobs.json 的 ref_images（通常 1 张场景图 + 1-2 张人物图），id 命名为 ep{集号}_镜{镜号}，复用出图阶段同一套生成脚本合成

这一镜的画面。

- 4 **逐张验收** —— 核对是否还原了画面描述、景别对不对、人物/场景是否与基准图保持一致；不合格就调整提示词重来，单镜连续 3 轮 选不出可用图就停下向用户汇报，不无限重试。
- 5 **登记归档** —— 每镜最终选中的一张整理进 `keyframes.md`：镜号 | 场景 | 出场人物 | 画面描述 | 选中文件 路径 | 分级。

归档位置：output/<剧名>/keyframes/ep0X/ 下的合成图片 + `keyframes.md`。这份表接下来会被 `video-gen` 直接引用作为图生视频的起始帧，如果之后手动改了文件名/位置，要同步回来更新这份表，否则下一步会引用 到失效路径。

05 归档目录规范

整条链路的归档全部挂在同一个根目录下，命名规则固定，方便随时回头找文件。下面是一个真实项目（《出狱后我成为了非洲矿王》）的实际目录结构，节选并加了注释：

```
output/<剧名>/
├─ storyboard/                                # 阶段②：分镜设计蓝本
│   ├── style_bible.md                       # 画风锚点 + 叙事基调
│   ├── characters.md                        # 人物形象提示词
│   ├── scenes.md                           # 场景背景提示词
│   └─ ep01.md ... ep10.md                  # 逐集分镜表
├─ assets/                                    # 阶段③：资产库
│   ├── jobs_round1.json ...                # 各批次出图任务清单
│   ├── SC01_监狱门口/
│   │   ├── SC01_监狱门口_00.png
│   │   └─ SC01_监狱门口_01.png            # 一个 job_id 一个子文件夹
│   ├── 江砚_出狱期_正面/ ...              # 角色基准图（同理，一版一图）
│   ├── selected.md                         # 资产库索引：最终入选路径登记表
│   └─ manifest.append.jsonl               # 每张图的生成记录（prompt/参考图/时间戳）
├─ keyframes/                                # 阶段④：关键帧图库
│   └─ ep01/
│       ├── ep01_镜03_00.png
│       └─ keyframes.md                    # 关键帧登记表
└─ videos/                                   # 阶段⑤起：视频编排/生成
    └─ ep01/
        ├── video_jobs.md                  # 提交清单（S/A级）
        ├── video_jobs.json                # 自建自制微调模型分支：结构化打包
        └─ kenburns_jobs.json              # B级：本地零成本运镜
```

三条通用规则，适用于每一层目录：

- 一个 **job_id** 一个子文件夹，文件名固定形如 <job_id>_00.png、_01.png.....重复调用 同一个 job 会接着编号往后生成，不覆盖已有文件。
- 每一层都有一张“登记表”（selected.md / keyframes.md / video_jobs.md），废弃的变体不删（留着方便回溯），但只有登记表里的路径才算“正式入选”，下游只认登记表，不会自己去猜哪张最新。
- 登记表 = 下一阶段的唯一输入——手动改文件名/挪位置 之后一定要同步回登记表，否则下一环节会引用到一个已经不存在的路径。

06 术语表

画风锚点

在分镜设计最开始定下的一句视觉风格描述（比如"现代都市剧写实照片 质感、电影感打光、竖屏构图"），后续所有角色/场景/镜头提示词开头必须 原样复用，防止中途风格跑偏。

S/A/B/C 分级

按镜头"动作需求等级"分档，决定花多少钱生成：S=高光动作镜（图生 视频/精细做），A=对话反应镜（短时长动效），B=过渡/空镜（本地运镜特效，零成本），C=复用镜（直接沿用已有素材）。

三视图

角色出图固定采用的模式：一张图里并排放正面、侧面、背面三个视角，同一站姿同一表情同一套服装，作为该角色贯穿全剧的基准参考图。

selected.md

资产库的索引表：登记每个角色/场景每套造型/时段"最终选中"的图片 文件路径，是关键帧生成阶段查找基准图的唯一依据。

ref_images

jobs.json 里的字段，填要参考哪几张已有图片来保持 "同一张脸/同一个场景"；留空表示纯文生图，不参考任何已有图。

manifest.append.jsonl

出图脚本自动追加写入的生成日志，每行一张图，记录当时用的 prompt、参考图、时间戳，用于回溯"这张图当初是怎么生成出来的"。

jobs.json

喂给出图脚本的任务清单，每条记录一个 id（决定输出 子文件夹名）、完整 prompt、要生成几张 count、参考图 ref_images。

制作流程说明 · 对应技能位于 .claude/skills/short-drama-*/，各技能的 SKILL.md 是本文档信息的原始来源。